

Guia Prático — Laravel MVC

Projeto ArenaPlay (trabalhoprogweb) · Referência rápida para criar novas funcionalidades

1. O ciclo de uma requisição (MVC)

Quando alguém acessa uma URL, o caminho percorrido é sempre este:

Navegador → Rota (web.php) → Controller → Model (banco) → View (HTML) → Navegador

Cada peça tem **uma responsabilidade**:

Peça	Onde fica	Responsabilidade
Rota	routes/web.php	Diz qual URL chama qual método do controller
Controller	app/Http/Controllers/	Recebe a requisição, decide a lógica, busca dados, devolve uma view
Model	app/Models/	Conversa com o banco de dados (uma tabela = um model)
View	resources/views/	O HTML que o usuário vê (Blade)

2. O MODEL — a tabela como objeto

Representa uma tabela do banco. Dois conceitos-chave:

- `$fillable` — lista de colunas que podem ser preenchidas de uma vez (proteção contra "mass assignment"). Se a coluna não estiver aqui, é ignorada ao salvar.
- Relacionamentos** — métodos que ligam tabelas sem escrever SQL.

```
class Arena extends Model
{
    protected $fillable = [
        'owner_id', 'name', 'description', 'address', 'phone', 'contact_email'
    ];

    // uma arena pertence a um proprietário
    public function owner()
    {
        return $this->belongsTo(Owner::class);
    }
}
```

Relacionamento	Significado	Como usar
<code>hasMany</code>	Um tem vários (Owner tem várias Arenas)	<code>\$owner->arenas</code>
<code>belongsTo</code>	Um pertence a outro (Arena pertence a Owner)	<code>\$arena->owner</code>
<code>hasOne</code>	Um tem exatamente um (User tem um Owner)	<code>\$user->owner</code>

3. O CONTROLLER — o cérebro

Recebe a requisição e decide o que fazer. Busca dados no Model e entrega para a View. Nunca escreve HTML nem SQL direto. Métodos padrão (convenção REST):

Método	Papel
<code>index()</code>	Listar todos os registros
<code>create()</code>	Mostrar o formulário de criação
<code>store()</code>	Salvar o que o formulário enviou
<code>show(\$id)</code>	Exibir um registro específico
<code>edit(\$id)</code>	Mostrar o formulário de edição
<code>update(\$id)</code>	Salvar as alterações
<code>destroy(\$id)</code>	Excluir um registro

```
public function store(Request $request)
{
    // 1. valida a entrada
    $validated = $request->validate([
        'nome' => ['required', 'string', 'max:120'],
        'endereco' => ['required', 'string', 'max:255'],
    ]);

    // 2. salva no banco (Model)
    Arena::create([
        'name' => $validated['nome'],
        'address' => $validated['endereco'],
    ]);

    // 3. redireciona
    return redirect()->route('arenas.index');
}
```

4. A VIEW — o HTML (Blade)

Ficam em `resources/views/` e usam Blade (HTML com `@` e `{{ }}`).

Sintaxe	O que faz
<code>{{ \$variavel }}</code>	Imprime um valor (com escape de segurança)
<code>@foreach ... @endforeach</code>	Laço de repetição
<code>@if ... @endif</code>	Condicional
<code>@extends('layouts.main')</code>	Usa um layout base
<code>@section('content')</code>	Preenche um "buraco" do layout
<code>{{ route('arenas.create') }}</code>	Gera a URL de uma rota pelo nome
<code>@csrf</code>	Token de segurança (obrigatório em todo formulário POST)
<code>@method('PUT')</code>	Simula PUT/DELETE (HTML só tem GET e POST)

```

@extends('layouts.main')

@section('content')
    @foreach($arenas as $arena)
        <h5>{{ $arena->name }}</h5>
        <a href="{{ route('arenas.edit', $arena->id) }}">Editar</a>
    @endforeach
@endsection

```

Atenção: os nomes das colunas no banco são em inglês (`name` , `address`). Os nomes em português (`nome` , `endereço`) só existem nos campos do *formulário*. Ao imprimir dados na view, use `$arena->name` , não `$arena->nome` .

5. A ROTA — liga a URL ao Controller

Toda rota tem 3 partes + um nome:

```
Route::get('/registerArenaOwners', [RegisterArenaOwnerController::class, 'create'])
//      metodo HTTP   |   URL       |   controller e metodo
      ->name('register.arena.owners'); // apelido da rota
```

Método	Quando usar
<code>Route::get()</code>	Abrir/exibir uma página (clicar em link)
<code>Route::post()</code>	Enviar formulário / criar algo
<code>Route::put()</code> / <code>patch()</code>	Atualizar
<code>Route::delete()</code>	Excluir

Atalhos importantes

Route::resource — cria as 7 rotas do CRUD de uma vez:

```
Route::resource('arenas', ArenaController::class);
```

Grupo com middleware — exige login para um conjunto de rotas:

```
Route::middleware(['auth'])->group(function () {
    Route::resource('arenas', ArenaController::class);
});
```

Os 3 arquivos de rota: `web.php` (páginas no navegador), `api.php` (API JSON), `console.php` (comandos de terminal).
Para páginas do site, é sempre `web.php`.

6. RECEITA: adicionar uma nova funcionalidade

Siga sempre estes 4 passos, nesta ordem:

1. **Rota** — declare em `web.php` (`get` para exibir, `post` para salvar) e dê um nome com `->name(...)`. Coloque dentro do grupo `middleware(['auth'])` se exigir login.
2. **Controller** — crie o método: busca dados (Model) + valida + decide + devolve view ou redirect.
3. **Model** — garanta que as colunas estão no `$fillable` e crie relacionamentos se precisar.
4. **View** — crie o arquivo `.blade.php` que exibe os dados / o formulário.

Exemplo completo: cadastro de proprietário (User + Owner)

```
// Controller: cria usuario E proprietario numa transacao
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => ['required', 'string', 'max:100'],
        'email' => ['required', 'email', 'unique:users,email'],
        'password' => ['required', 'confirmed', 'min:8'],
        'company_name' => ['required', 'string', 'max:150'],
        'tax_id' => ['required', 'string', 'unique:owners,tax_id'],
    ]);

    $user = DB::transaction(function () use ($validated) {
        $user = User::create([
            'name' => $validated['name'],
            'email' => $validated['email'],
            'password_hash' => Hash::make($validated['password']),
            'type' => 'owner',
        ]);
        Owner::create([
            'user_id' => $user->id,
            'company_name' => $validated['company_name'],
            'tax_id' => $validated['tax_id'],
        ]);
        return $user;
    });

    Auth::login($user);
    return redirect()->route('owners.dashboard');
}
```

Por que a transação? `DB::transaction` garante atomicidade: se criar o Owner falhar, o User não fica salvo pela metade. Ou cria os dois, ou nenhum.

7. Comandos artisan úteis

Comando	O que faz
<code>php artisan serve</code>	Sobe o servidor local (http://127.0.0.1:8000)
<code>php artisan route:list</code>	Lista todas as rotas registradas
<code>php artisan make:controller NomeController</code>	Cria um controller
<code>php artisan make:model Nome -mc</code>	Cria model + migration + controller juntos
<code>php artisan make:migration create_x_table</code>	Cria uma migration
<code>php artisan migrate</code>	Roda as migrations (cria as tabelas)
<code>php artisan migrate:fresh</code>	Apaga tudo e recria o banco (cuidado!)
<code>php artisan tinker</code>	Console interativo para testar Models

8. Erros comuns e como evitar

Sintoma	Causa provável
404 ao clicar num link	A rota não existe em web.php (ou o nome está errado)
419 Page Expired ao enviar form	Faltou <code>@csrf</code> no formulário
Campo aparece em branco na view	Nome da coluna errado (ex.: <code>\$x->nome</code> em vez de <code>\$x->name</code>)
Dado não salva no banco	Coluna faltando no <code>\$fillable</code> do Model
Erro 403 Forbidden	Middleware barrou (ex.: precisa estar logado ou ser admin)
Erro 500 ao enviar form	Faltou validação; valor inválido chegou ao banco